Assignment #2. Advanced CSS (Flexbox & Grid)

Objective:
By the end of this assignment, students will be able to:

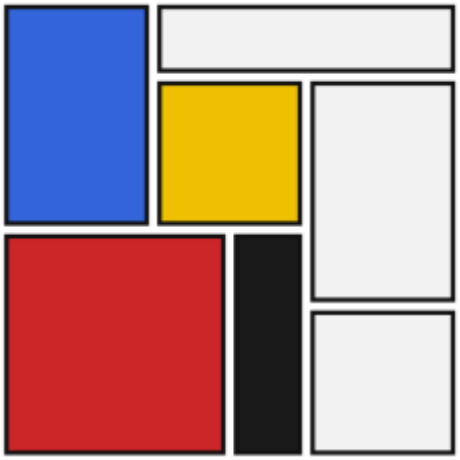
- use selectors, colors, fonts and spacing consistently across a set of pages;
- solve real-world layout challenges by making deliberate design decisions between Flexbox and CSS Grid;
- establish foundational responsive design using fluid CSS properties.

Rules for Submission:

1. This is an individual assignment. If the works of different students are the same, the grade is **0**.
2. Submit before the deadline. **Late works are not accepted!** To get a grade you must **submit before the deadline AND defend your work at the practice lesson**. Submission without a defense will result in a grade of 0.
3. You must submit: your report in **Assignment2_YourName.docx** (or .pdf) format. If you submit in other formats, your grade will be **0**.
4. No Bootstrap, no JavaScript and no media queries in this assignment.
5. IN THE REPORT MUST BE:
   - Your name and group
   - Every task with screenshots of the result and of the key CSS
   - A short explanation in your own words of how you solved each task
   - A short conclusion about what you learned
   - The **GitHub repository URL** with the source code.

Task 1. Geometric composition (CSS Grid)

Reproduce the composition below with a single CSS Grid. It looks simple, but the blocks are of different sizes, so you have to work out from the drawing how many columns and rows each of them spans (6 columns x 6 rows).



Your composition should:

— repeat the drawing: the same blocks in the same places, with the same proportions;
— stay square and scale as a whole with the window, never growing taller than the screen;
— get the black lines from the gap of the grid, not from borders or margins of the blocks.

Tools: Use CSS Grid only. Floats, absolute positioning and fixed sizes are not allowed.

Task 2. Component library (Flexbox)

Build a small library of the components every site is assembled from. Each one is a separate block on the same page, and each has to survive content it was not designed for.

Your library should include:

(1) a navigation bar with a logo, links and a group of buttons that stays at the right edge however long the links become;
(2) a row of cards with descriptions of different length, where all the cards are equally tall and all the buttons sit on one line, without any fixed height;
(3) a pagination bar with the page numbers in the center, an arrow on either side of them, and the result count at the right edge (on a narrow screen the parts may wrap);
(4) a comment block where the text wraps beside the avatar and never slides under it;
(5) a pricing table of three plans where the recommended one is taller than its neighbors, again without a fixed height.

The five components share one stylesheet and one set of colors, spacing and fonts, so that the page looks like one library and not five exercises.
Tools: Use Flexbox only. No Grid, no floats, and no fixed widths or heights for the layout (the avatar may keep its own size).

Task 3. Three layouts, one markup (Grid and Flexbox)

Write a page with a dozen articles or products, and then make the same content display in three ways: (1) a grid of cards, (2) a compact list, and (3) a magazine front page with one lead item. Switching between them is done by changing a single class on the container, and the HTML stays untouched.

(1) In the grid mode the number of columns follows the window width.
(2) In the list mode every item becomes a row with the picture on the left, the title and the text beside it, and the date at the right edge of the row.
(3) In the magazine mode one item spans several columns and rows while the others fill in around it.

Give the items titles and descriptions of different length: all three modes must survive that, and all three must hold from a narrow window to a wide one without media queries.
Show all three modes in the report and say in one sentence which CSS tool does the work in each of them.
Tools: Grid for the three modes, Flexbox inside the item.

Assessment criteria

Criterion	What is checked	Points
Selectors, colors, fonts, spacing	- selectors are used efficiently in external stylesheets - a cohesive color palette, proper typography and consistent spacing across the work - the result looks considered and reads well	**15**
Flexbox and Grid implementation	- Flexbox and Grid are applied to the problems they belong to, and each task behaves the way it is described in the assignment - layout is produced by Flexbox and Grid themselves, not by fixed sizes, manual offsets or positioning	**15**
Responsive design	- the pages hold up on both small and large screens under content of different length, without frameworks and without media queries.	**15**

Criterion	What is checked	Points
Defense	- explains the work the student has done and demonstrates the pages working live without errors - answers the instructor's questions about their own code and the topic - completes the live coding task independently, without assistance or hints	**40**
Report	- contains everything the assignment asks for, in the order of the tasks; - the explanations are written in the student's own words and show what was actually done; - the report is well structured and looks like a finished document: easy to navigate, with screenshots that can actually be read; - ends with a short conclusion about what the student learned	**15**
Total		**100**

Resources:

1. Abitova G.A. Web technologies Front-End Development. Part 1 (2022).
2. https://www.w3schools.com/css/css3_flexbox.asp
3. https://www.w3schools.com/css/css_grid.asp
4. https://learn.astanait.edu.kz

***Questions will be based on the provided resources**

Good luck!